18 B 樹

Part V Advanced Data Structures

本章定位

B-tree在一個頁內放多個鍵，降低搜尋路徑的磁碟頁數。插入與刪除的難點是搬鍵時同步搬child指標，保住全域鍵序、每節點容量及所有葉同深度。本章分別計算頁I/O與頁內CPU工作。

前置知識

多叉搜尋樹、磁碟 I/O 與高度平衡。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以最小度 t 描述 B-tree 的節點容量。
- 分析搜尋、插入分裂與刪除合併的 I/O。
- 檢查根節點與鍵數的邊界。
- 18.1 Definition of B-trees
- 18.2 Basic operations on B-trees
- 18.3 Deleting a key from a B-tree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

18.1 Definition of B-trees

B-tree 把多個排序鍵與多個子指標放在同一個節點，通常讓一節點對應一個磁碟頁。本章假設鍵互異，每鍵的資料或記錄指標隨鍵一起搬移；重複鍵插入回報已存在。這是內節點也存資料的 B-tree，與只在葉存資料、父節點複製 separator 的 B+ tree 不同。

最小度 t 是整數且 t≥2。每個非根節點有 t−1..2t−1 個鍵；非空根有1..2t−1個鍵。具有k個鍵的內節點恰有k+1個child，且所有葉深度相同，記為高度h，根深度0。

節點鍵 a0<a1<⋯<a_(k−1) 把child的全域範圍分開：child0的所有鍵<a0，childi的鍵在(a_(i−1),a_i)，childk的所有鍵>a_(k−1)。只檢查每節點內排序不夠，還必須檢查整棵子樹範圍。

空樹以零鍵的葉根表示，高度0；高度上界定理只用在n≥1的樹。

高度上界與下界

最少鍵的非空高度h樹，根只有1鍵且2個child，其餘節點恰有t−1鍵。深度i≥1至少有2t^(i−1)個節點，因此

$n \geq 1+\Sigma_{i=1}^{h} 2t^{i-1}(t-1)=2t^h-1$，

故 $h \leq \log_t((n+1)/2)$。h=0時空和為0，同樣成立。

相反，每節點至多2t−1鍵、最多2t個child，所以

$n \leq (2t-1)\Sigma_{i=0}^{h}(2t)^i=(2t)^{(h+1)}-1$，

故 $h \geq \log_{(2t)}(n+1)-1$。兩界分別控制最稀疏與最密集的樹，不能混用來猜「通常幾層」。

十億個鍵的頁數界

取t=100、n=10^9。高度2最多200^3−1=7,999,999鍵，不夠；高度3最多200^4−1=1,599,999,999鍵，容得下，所以最小可能高度3。

高度4最少2×100^4−1=199,999,999鍵，仍可；高度5最少19,999,999,999鍵，太多，所以最大可能高度4。因此任意符合條件的樹h∈{3,4}，搜尋走4或5個節點。

若根已常駐記憶體且其他頁起初不在快取，這是3或4次頁讀取；根也不在記憶體則是4或5次。單節點內用線性掃描要O(t)比較，故CPU O(th)；搜尋可改binary search為O(h log t)比較，但插入/刪除的陣列搬移仍可能O(t)每層。

每頁可容納的t由頁大小、key/record/child指標大小與metadata決定。I/O次數不是硬碟延遲測量，也不能直接用鍵比較次數代替。

18.2 Basic operations on B-trees

節點陣列採0-based。搜尋k時找第一個大於等於k的鍵位置i；若不存在令i=鍵數。

```
search(x,k):
    i = first index with x.keys[i]>=k, or len(x.keys)
    if i<len(x.keys) and x.keys[i]==k: return (x,i)
    if x is leaf: return NOT_FOUND
    read page x.children[i]
    return search(x.children[i],k)
```

若沒有相等鍵，節點分隔性保證至多一個child可能含k；若它也不存在，其餘child全被鍵序排除。每步降低剩餘高度，最後在葉成功或失敗。用於插入前的重複鍵檢查最多另花一條搜尋路徑，不改變漸近I/O界。

分裂滿子節點的全部移接

前提為父p未滿、y=p.children[i]有2t−1鍵。先保存中位鍵median=y.keys[t−1]，再分割：

```
z.keys = y.keys[t .. 2t-2]
y.keys = y.keys[0 .. t-2]
if y is internal:
    z.children = 原y.children[t .. 2t-1]
    y.children = 原y.children[0 .. t-1]
p.keys.insert(i,median)
p.children.insert(i+1,z)
write y,z,p
```

左右各t−1鍵；若為內節點，各有t個child。median從子節點移出成為父的真實鍵，不在兩邊重複保留。各child原來依median分成左右範圍，因此排序成立，葉深度不變。父未滿保證加1鍵仍≤2t−1。

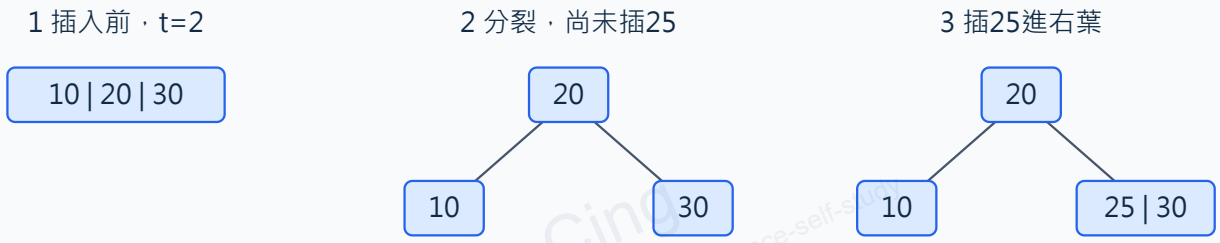
插入前先分裂，保持下行節點未滿

```
insert(k):
    if search(root,k) succeeds: return EXISTS
    if root is full:
        root = new empty internal node with old root as child0
        split(root,0)
    x=root
    while x is internal:
        i = child index selected by k
        if x.children[i] is full:
            split(x,i)
            if k>x.keys[i]: i++
        x=x.children[i]
    insert k into x.keys in sorted position
```

迴圈不變量是x未滿。若下一child滿，當場分裂後選一個t−1鍵的child，因此下次仍未滿；葉插入最多加1，不溢位。每層至多一次split，唯獨根分裂會增加高度，且所有葉同時增加一層。

插入前先分裂，保持下行節點未滿 （續）

滿根分裂，再插入25的三態



中位鍵20移到父，原葉不再存20；25>20，所以到右葉插入。

根分裂的數值核驗

t=2時滿葉根[10,20,30]，插入25：

階段	根	children	全部鍵的中序
插入前	[10,20,30]葉	無	10,20,30
先分裂	[20]	[10]、[30]	10,20,30
插25	[20]	[10]、[25,30]	10,20,25,30

20只出現在根，並沒有留在葉。插入25時因25>20選child1，該葉原有1鍵，插入後2鍵仍≤3。兩葉深度都是1。

split的頁寫入為舊child、新child、父，共常數次；父與舊child的讀取是否已在快取必須明定。O(h)表示隨樹高線性，不是所有插入固定恰有相同頁讀寫數。

18.3 Deleting a key from a B-tree

刪除採較強下行條件。除根外，每次遞迴進入的節點先有至少t鍵，因此沿路失去1鍵後仍有t−1鍵。根可以在最後由0鍵內節點縮成唯一child。

找到鍵k時分三種：

1. x是葉，直接刪k。
2. x是內節點，k左child至少t鍵，則以左子樹最大鍵pred取代k，再在左子樹刪pred。若左child只有t−1而右child至少t鍵，對稱改用successor。
3. 兩child都只有t−1鍵，把左child、父鍵k、右child合成2t−1鍵節點，再遞迴在合成節點刪k。

不能直接從內節點陣列移除k而保留全部child，因鍵數與child數及區間分隔都會失配。用pred/successor替換時，記錄資料也隨鍵移動。

還沒找到鍵時，先修復將下降的Child

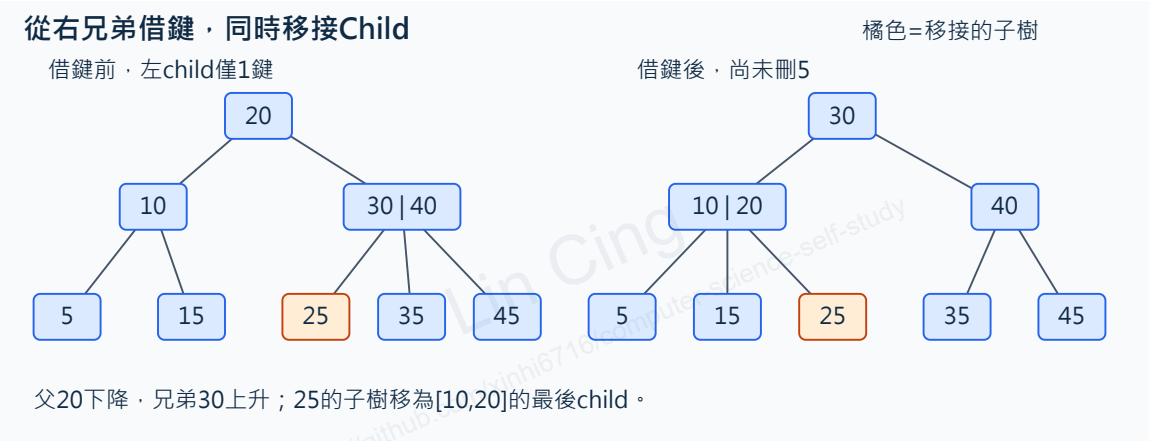
若x尚無k，依搜尋規則決定child y。若y已有至少t鍵，直接下降；否則y只有t−1鍵：

- 若相鄰左兄弟至少t鍵，父分隔鍵下降成y的第一鍵，左兄弟最大鍵上升補父；內節點還要把左兄弟最右child移成y的第一child。
- 否則若右兄弟至少t鍵，父分隔鍵下降成y的最後鍵，右兄弟最小鍵上升補父；內節點還要把右兄弟最左child移成y的最後child。
- 否則與任一相鄰兄弟及兩者間父鍵合併。合併後有2t−1鍵，再對該節點下降；若合併左兄弟，搜尋child索引須減1。

借鍵使兩個child都保持下限，父鍵數不變；合併使父少1鍵、少1個child，child鍵序與等深度仍成立。不存在的k也可能沿路觸發合法重整，但最後回報NOT_FOUND，鍵集合不能改變。

從右兄弟借鍵時，Child指標也要搬

以下t=2，原樹搜尋刪除5時，根的左child只有1鍵，不可直接下降；右child有2鍵，可借。



操作後根30，左child[10,20]有三個葉[5]、[15]、[25]；右child[40]有[35]、[45]。25這棵子樹不能留在右側，因25<新父30且20<25，必須是[10,20]的最後child。

從右兄弟借鍵時，Child指標也要搬 (續)

這張圖是下降前的修復，尚未刪5。接著在左child內，待降葉[5]與右兄弟[15]都只有1鍵，所以把[5]、父鍵10、[15]合成[5,10,15]，再刪5得[10,15]。左內節點剩[20]，children為[10,15]、[25]；根30及右子樹不變。所有葉仍深度2。

內節點替換與根縮高的數例

若根[20]、左葉[5,10]、右葉[30]，刪20採前驅10。新根[10]、左葉[5]、右葉[30]，中序為5,10,30。不是在父複製10後忘了從左葉刪掉它。

若根[20]、左葉[5]、右葉[30]，兩邊皆只有t−1鍵，不能任意拿走一邊的鍵。先合成[5,20,30]，舊根暫時為空且僅一child；遞迴在合成葉刪20得[5,30]，最後用這個唯一child取代空內根。

內節點替換與根縮高的數例 (續)



只有根空了才能縮高；一般內節點不可以把唯一child直接接到祖父，否則會破壞所有葉同深度。最後一個鍵刪掉時保留零鍵葉根，代表空樹。

刪除偽碼與正確性歸納

```
delete(x,k):
    i = first key >=k, or key count
    if key i equals k:
        if leaf: erase key i; return FOUND
        if child i has >=t keys: replace k by pred; delete(child i,pred)
        elif child i+1 has >=t keys: replace k by succ; delete(child i+1,succ)
        else: y=merge(child i,key i,child i+1); delete(y,k)
        return FOUND
    if leaf: return NOT_FOUND
    y=child i
    if y has t-1 keys: borrow from a rich neighbor, else merge as above
    return delete(the repaired target child,k)
after top call: if root has0keys and1child: root=its child
```

以剩餘高度歸納。葉刪除在下行條件下不低於t−1；內部替換保持鍵序與總鍵集合只少目標k；借鍵/合併先保證子問題合法且高度下降。非根x進入時至少t鍵，合併使x少1仍合法；根例外由最後縮高處理。這同時證明終止、鍵集合正確與結構不變量。

操作成本與實作檢查

每次下降只處理常數個相鄰節點，頁讀寫O(1)每層，所以搜尋、插入、刪除皆O(h+1)=O(1+log_t(n+1))頁存取。節點內搬移鍵/child最多O(t)，CPU O(t(h+1))。取得pred或succ若分開搜尋再刪，至多增加同一子樹的常數次路徑；也可合併成一次下降。

這些界假設一節點可放一頁且根快取模型固定，未含併發鎖、交易日誌、頁壓縮與碎片管理。它們是額外實作契約，不能從抽象B-tree直接推得。

每次操作後可檢查鍵全域範圍、非根鍵數下限、child數=鍵數+1、所有葉同深度、無重複child指標、總中序與參考集合相同。有限程式檢查能抓實作錯誤，前面的歸納才給任意合法操作的正確性。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 18。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study